

ROS 2: robot modeling

Dawid Seredyński

Warsaw University of Technology

June 2026

Aim of this lecture

To introduce math for modeling of manipulators

To explain direct kinematics

To introduce *ROS 2*–standard unified robot description format (URDF)

To show *ROS 2* tools used for transformations

Learning goals

By the end of this module, you should be able to:

- explain what is a pose, rotation, translation, transformation
- explain what is a link, joint, kinematic chain
- build your own robot model using URDF
- use TF to get transformation between two frames
- use TF to broadcast transformation

In this course manipulation is considered

However, robot modeling in ROS 2 applies to mobile robots as well

REPs (Robotics Enhancement Proposals), e.g.:

- REP-103: Standard Units of Measure and Coordinate Conventions★
- REP-105: Coordinate Frames for Mobile Platforms★
- REP-120: Coordinate Frames for Humanoid Robots★

Sciavicco, L., & Siciliano, B. (2012).
Modelling and control of robot manipulators.
Springer Science & Business Media.★

- an excellent book
- available for free

Manipulator

A manipulator:

- a kinematic chain of rigid bodies (*links*)
- connected by means of revolute or prismatic *joints*

Two ends (*links*):

- base
- end effector

End effector motion: composition of the elementary motions between *links*

Kinematic structures:

- open-chain
- closed-chain
- tree-like

A rigid body is completely described in space by its *position* and *orientation* (i.e. *pose*) with respect to a reference frame

O — xyz – the orthonormal reference frame

$\mathbf{x}, \mathbf{y}, \mathbf{z}$ – the unit vectors of the frame axes

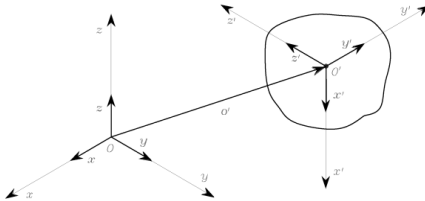


image source: <https://link.springer.com/book/10.1007/978-1-4471-0449-0>

$\mathbf{o}'$ – position of a point O' on the rigid body wrt. O — xyz :

$$\mathbf{o}' = o'_x \mathbf{x} + o'_y \mathbf{y} + o'_z \mathbf{z}, \quad \text{compact form: } \mathbf{o}' = \begin{bmatrix} o'_x \\ o'_y \\ o'_z \end{bmatrix}$$

$\mathbf{o}'$ is a bound vector

$O'-x'y'z'$ – orthonormal frame attached to the body with origin in O' and x', y', z' – unit vectors of the frame axes

$$\mathbf{x}' = x'_x \mathbf{x} + x'_y \mathbf{y} + x'_z \mathbf{z}$$

$$\mathbf{y}' = y'_x \mathbf{x} + y'_y \mathbf{y} + y'_z \mathbf{z}$$

$$\mathbf{z}' = z'_x \mathbf{x} + z'_y \mathbf{y} + z'_z \mathbf{z}$$

$\mathbf{x}', \mathbf{y}', \mathbf{z}'$ are orthogonal, unit vectors

$$\mathbf{x}'^T \mathbf{y}' = 0, \quad \mathbf{y}'^T \mathbf{z}' = 0, \quad \mathbf{z}'^T \mathbf{x}' = 0$$

$$\mathbf{x}'^T \mathbf{x}' = 1, \quad \mathbf{y}'^T \mathbf{y}' = 1, \quad \mathbf{z}'^T \mathbf{z}' = 1$$

Rotation matrix:

$$\mathbf{R} = \begin{bmatrix} \mathbf{x}' & \mathbf{y}' & \mathbf{z}' \end{bmatrix} = \begin{bmatrix} x'_x & y'_x & z'_x \\ x'_y & y'_y & z'_y \\ x'_z & y'_z & z'_z \end{bmatrix}$$

$\mathbf{R}$ is orthogonal matrix, thus

$$\mathbf{R}^T \mathbf{R} = \mathbf{I}, \quad \mathbf{I} \text{ is identity matrix}$$

and

$$\mathbf{R}^T = \mathbf{R}^{-1}$$

Elementary rotations

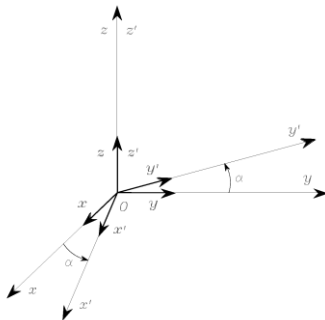


image source: <https://link.springer.com/book/10.1007/978-1-4471-0449-0>

$$R_z(\alpha) = \begin{bmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad R_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix} \quad R_x(\gamma) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \gamma & -\sin \gamma \\ 0 & \sin \gamma & \cos \gamma \end{bmatrix}$$

$$R_k(-\vartheta) = R_k^T(\vartheta) \quad k = x, y, z$$

Rotation of vector

If origins of $O\text{--}xyz$ and $O'\text{--}x'y'z'$ coincide: $\mathbf{o}' = \mathbf{0}$

Position of a point P :

$$\mathbf{p} = \begin{bmatrix} p_x \\ p_y \\ p_z \end{bmatrix} \quad \text{wrt. } O\text{--}xyz$$

$$\mathbf{p}' = \begin{bmatrix} p'_x \\ p'_y \\ p'_z \end{bmatrix} \quad \text{wrt. } O'\text{--}x'y'z'$$

$$\mathbf{p} = p'_x \mathbf{x}' + p'_y \mathbf{y}' + p'_z \mathbf{z}' = \begin{bmatrix} \mathbf{x}' & \mathbf{y}' & \mathbf{z}' \end{bmatrix} \mathbf{p}'$$

$$\mathbf{p} = R\mathbf{p}' \quad \mathbf{p}' = R^T \mathbf{p}$$

Composition of rotations

Three frames with a common origin O :

$$O-x_0y_0z_0, O-x_1y_1z_1, O-x_2y_2z_2$$

The vector $\mathbf{p}$ expressed wrt. above frames:

$$\mathbf{p}^0, \mathbf{p}^1, \mathbf{p}^2$$

R_i^j – the rotation matrix of Frame i with respect to Frame j ; $R_i^j = (R_j^i)^{-1} = (R_j^i)^T$

$$\mathbf{p}^1 = R_2^1 \mathbf{p}^2$$

$$\mathbf{p}^0 = R_1^0 \mathbf{p}^1$$

$$\mathbf{p}^0 = R_2^0 \mathbf{p}^2$$

thus:

$$R_2^0 = R_1^0 R_2^1 \quad (\text{composition of successive rotations})$$

Euler Angles

$\mathbf{R}$ has 9 elements and 6 constraints (orthogonality and unit length of column vectors of $\mathbf{R}$)

3 parameters are sufficient to describe orientation (minimal representation)

Euler Angles: 3 angles $\phi = [\varphi \ \vartheta \ \psi]^T$ for composition of 3 rotation matrices along coordinate axes:

- two successive rotations – not along the same axis
- total 12 combinations (XYX, XYZ, XZX, XZY, YXY, YXZ, etc.)
- popular: ZYZ, ZYX (or Roll-Pitch-Yaw)

Euler Angles ZYZ: composition of rotations wrt. current frames ZYZ

$$\mathbf{R}(\phi) = \mathbf{R}_z(\varphi)\mathbf{R}_{y'}(\vartheta)\mathbf{R}_{z''}(\psi) = \begin{bmatrix} c_\varphi c_\vartheta c_\psi - s_\varphi s_\psi & -c_\varphi c_\vartheta s_\psi - s_\varphi c_\psi & c_\varphi s_\vartheta \\ s_\varphi c_\vartheta c_\psi + c_\varphi s_\psi & -s_\varphi c_\vartheta s_\psi + c_\varphi c_\psi & s_\varphi s_\vartheta \\ -s_\vartheta c_\psi & s_\vartheta s_\psi & c_\vartheta \end{bmatrix}$$

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

Inverse problem, for $\vartheta \in (0, \pi)$:

$$\varphi = \text{Atan2}(r_{23}, r_{13})$$

$$\vartheta = \text{Atan2}(\sqrt{r_{13}^2 + r_{23}^2}, r_{33})$$

$$\psi = \text{Atan2}(r_{32}, -r_{31})$$

What if $\vartheta = 0, \pi$?

RPY = Roll, Pitch, Yaw angles

In *ROS 2* / URDF convention:

- roll: rotation about the fixed X axis
- pitch: rotation about the fixed Y axis
- yaw: rotation about the fixed Z axis

$$\mathbf{R} = \mathbf{R}_z(\text{yaw}) \mathbf{R}_y(\text{pitch}) \mathbf{R}_x(\text{roll})$$

This can be described equivalently as:

- fixed-frame / extrinsic rotations in order: $X \rightarrow Y \rightarrow Z$
- current-frame / intrinsic rotations in order: $Z \rightarrow Y \rightarrow X$

Always specify the convention: axis order, fixed/current frame, and angle names.

Angle-axis representation (nonminimal, 4 parameters):

$$\mathbf{R}(\vartheta, \mathbf{r})$$

where $\mathbf{r}$ is a unit vector (axis) and ϑ is angle of rotation around $\mathbf{r}$

$$\mathbf{R}(\vartheta, \mathbf{r}) = \mathbf{R}(-\vartheta, -\mathbf{r})$$

When $\vartheta = 0$, $\mathbf{r}$ is arbitrary (singularity)

Unit Quaternion:

$$\mathcal{Q} = \{\eta, \epsilon\}$$

where:

$$\eta = \cos \frac{\vartheta}{2} \quad (\text{scalar part})$$

$$\epsilon = \sin \frac{\vartheta}{2} \mathbf{r} \quad (\text{vector part})$$

ϑ – angle, $\mathbf{r}$ – axis, unit length: $\eta^2 + \epsilon_x^2 + \epsilon_y^2 + \epsilon_z^2 = 1$

$$\mathcal{Q}^{-1} = \{\eta, -\epsilon\}$$

Quaternion corresponding to $\mathbf{R} = \mathbf{I}$ (identity):

$$\{1, \mathbf{0}\}$$

Quaternions

Advantages:

- no singularity
- compared to a 3x3 rotation matrix:
 - ◊ more compact
 - ◊ more efficient composition
 - ◊ better numerical properties
- good for interpolation – the `slerp` algorithm (Spherical linear interpolation)

Disadvantages:

- not intuitive for humans
- redundant
- sign ambiguity: Q and $-Q$ is the same rotation
- different conventions: $[x, y, z, w]$ or $[w, x, y, z]$

Coordinate Transformation

P - an arbitrary point in space

$\mathbf{p}^0$ - coordinates of P in frame $O_0-x_0y_0z_0$ (a bound vector)

$\mathbf{p}^1$ - coordinates of P in frame $O_1-x_1y_1z_1$

$\mathbf{o}_1^0$ - origin of frame 1 wrt. frame 0

$\mathbf{R}_1^0$ - rotation matrix of frame 1 wrt. frame 0

$$\mathbf{p}^0 = \mathbf{o}_1^0 + \mathbf{R}_1^0 \mathbf{p}^1 \quad (\text{coordinate transformation: translation + rotation})$$

Inverse transformation (premultiplied by $\mathbf{R}_0^1$):

$$\mathbf{p}^1 = -\mathbf{R}_0^1 \mathbf{o}_1^0 + \mathbf{R}_0^1 \mathbf{p}^0$$

Introduction

Robot modeling

Manipulator

Pose, transformation

Elementary rotations

Representation of
rotation

**Coordinate
Transformation**

Direct kinematics

DH

ROS 2
integration

URDF

tf2

Homogeneous Transformation

Homogeneous representations of vectors

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ 1 \end{bmatrix} \quad (\text{for a bound vector } \mathbf{p})$$

$$\tilde{\mathbf{p}} = \begin{bmatrix} \mathbf{p} \\ 0 \end{bmatrix} \quad (\text{for an unbound vector } \mathbf{p})$$

Homogeneous transformation matrix:

$$\mathbf{A}_1^0 = \begin{bmatrix} \mathbf{R}_1^0 & \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} x_1^0 & y_1^0 & z_1^0 & o_1^0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} x_{1,x}^0 & y_{1,x}^0 & z_{1,x}^0 & o_{1,x}^0 \\ x_{1,y}^0 & y_{1,y}^0 & z_{1,y}^0 & o_{1,y}^0 \\ x_{1,z}^0 & y_{1,z}^0 & z_{1,z}^0 & o_{1,z}^0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Transformation

$$\tilde{\mathbf{p}}^0 = \mathbf{A}_1^0 \tilde{\mathbf{p}}^1$$

$$\tilde{\mathbf{p}}^1 = \mathbf{A}_0^1 \tilde{\mathbf{p}}^0 = (\mathbf{A}_1^0)^{-1} \tilde{\mathbf{p}}^0$$

$$\mathbf{A}_0^1 = \begin{bmatrix} \mathbf{R}_1^{0T} & -\mathbf{R}_1^{0T} \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_0^1 & -\mathbf{R}_0^1 \mathbf{o}_1^0 \\ \mathbf{0}^T & 1 \end{bmatrix}$$

Orthogonality property does not hold, in general:

$$\mathbf{A}^{-1} \neq \mathbf{A}^T$$

Transformations can be composed:

$$\tilde{\mathbf{p}}^0 = \mathbf{A}_1^0 \mathbf{A}_2^1 \dots \mathbf{A}_n^{n-1} \tilde{\mathbf{p}}^n$$

Direct kinematics

Mechanical structure of a manipulator:

- a number of degrees of mobility which uniquely determine its configuration
- each degree of mobility:
 - ◊ associated with a joint articulation
 - ◊ constitutes a joint variable

Direct kinematics: compute the position and orientation of the end effector as a function of the joint variables

$$\mathbf{T}_n^0(\mathbf{q}) = \mathbf{A}_1^0(q_1)\mathbf{A}_2^1(q_2)\dots\mathbf{A}_n^{n-1}(q_n)$$

$\mathbf{q}$ – (n × 1) vector of joint variables

$\mathbf{A}_i^{i-1}(q_i)$ – a kinematic relationship between consecutive links $i - 1$ and i (for $i = 1, \dots, n$)

Introduction

Robot modeling

Manipulator

Pose, transformation

Elementary rotations

Representation of
rotation

Coordinate
Transformation

Direct kinematics

DH

ROS 2
integration

URDF

tf2

Base to end-effector:

$$\mathbf{T}_e^b(\mathbf{q}) = \mathbf{T}_0^b \mathbf{T}_n^0(\mathbf{q}) \mathbf{T}_e^n$$

$\mathbf{T}_0^b$ – a constant pose of frame 0 wrt. base frame

$\mathbf{T}_e^n$ – a constant pose of end-effector frame wrt. frame n

Denavit-Hartenberg Convention

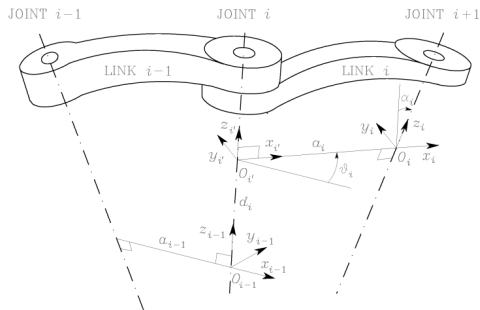


image source: <https://link.springer.com/book/10.1007/978-1-4471-0449-0>

axis i : axis of the joint connecting link $i - 1$ to link i

- z_i : along the axis of joint $i + 1$
- find common normal ($z_{i-1} \rightarrow z_i$)
- O_i : intersection of z_i with ($z_{i-1} \rightarrow z_i$); O'_i : intersection of z_{i-1} with ($z_{i-1} \rightarrow z_i$)
- x_i : along ($z_{i-1} \rightarrow z_i$) with direction from joint i to joint $i + 1$
- choose axis y_i so as to complete a right-handed frame

Denavit-Hartenberg Convention

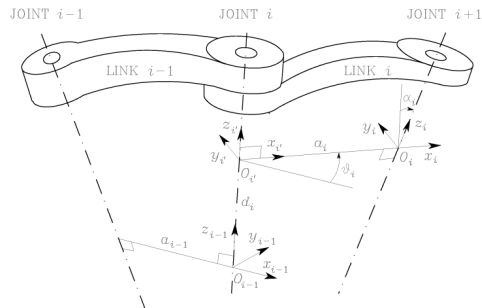


image source: <https://link.springer.com/book/10.1007/978-1-4471-0449-0>

Nonunique definition of the link frame, if:

- for frame 0: O_0, x_0 – arbitrary
- for frame n : z_n is not uniquely defined; typically – aligned with z_{n-1}
- two consecutive axes are parallel: common normal is not unique
- two consecutive axes intersect: x_i is arbitrary
- joint i is prismatic: direction of z_{i-1} is arbitrary

Denavit-Hartenberg Convention

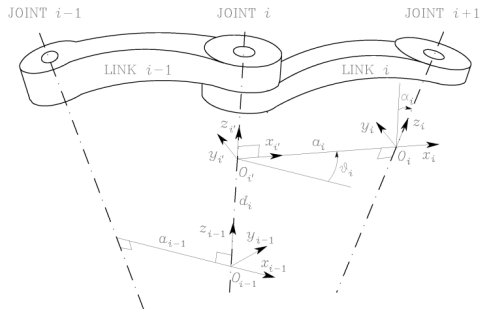


image source: <https://link.springer.com/book/10.1007/978-1-4471-0449-0>

Pose of link $i + 1$ wrt. link i is completely specified by:

- a_i : distance between O_i and $O_{i'}$ (constant)
- d_i : coordinate of $O_{i'}$ along z_{i-1} (variable, if the joint i is prismatic)
- α_i angle between z_{i-1} and z_i about x_i (constant)
- ϑ_i : angle between x_{i-1} and x_i about z_{i-1} (variable, if the joint i is revolute)

Pose of link i wrt. link $i-1$

- translate by d_i along z_{i-1} and rotate by ϑ_i about z_{i-1} :

$$\mathbf{A}_{i'}^{i-1} = \begin{bmatrix} c_{\vartheta_i} & -s_{\vartheta_i} & 0 & 0 \\ s_{\vartheta_i} & c_{\vartheta_i} & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- translate by a_i along $x_{i'}$ and rotate by α_i about $x_{i'}$

$$\mathbf{A}_i^{i'} = \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & c_{\alpha_i} & -s_{\alpha_i} & 0 \\ 0 & s_{\alpha_i} & c_{\alpha_i} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- result:

$$\mathbf{A}_i^{i-1}(q_i) = \mathbf{A}_{i'}^{i-1} \mathbf{A}_i^{i'} = \begin{bmatrix} c_{\vartheta_i} & -s_{\vartheta_i} c_{\alpha_i} & s_{\vartheta_i} s_{\alpha_i} & a_i c_{\vartheta_i} \\ s_{\vartheta_i} & c_{\vartheta_i} c_{\alpha_i} & -c_{\vartheta_i} s_{\alpha_i} & a_i s_{\vartheta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Unified Robot Description Format (URDF)★ – describes a robot

- links
 - ◇ name – identifier
 - ◇ collision geometry
 - ◇ visual geometry
 - ◇ inertial – mass, CoM, inertial parameters
- joints
 - ◇ name – identifier
 - ◇ type, e.g. revolute, prismatic
 - ◇ parent / child link
 - ◇ axis
 - ◇ limits
- additional parameters, e.g. for simulator

Introduction

Robot modeling

Manipulator

Pose, transformation

Elementary rotations

Representation of
rotation

Coordinate
Transformation

Direct kinematics

DH

ROS 2
integration

URDF

tf2

Joint, link

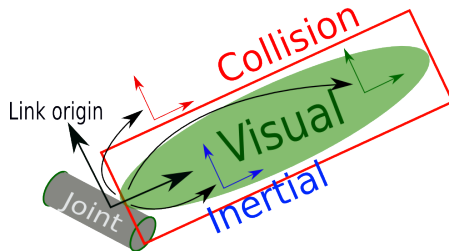
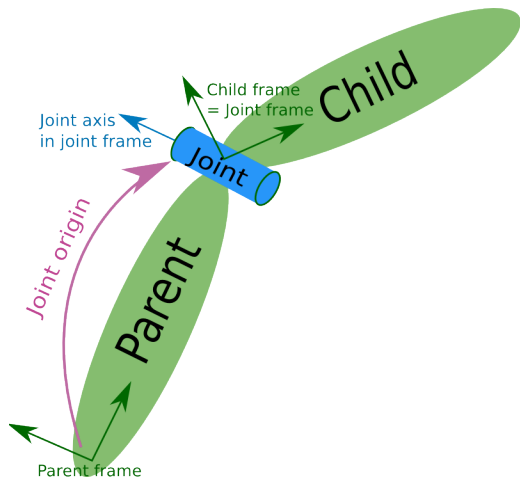


image source: <https://wiki.ros.org/urdf/XML>

Parametrized URDF: xacro

xacro★ – macro language for XML

- parameters
- macros
- expressions
- conditions (if, unless)
- includes

Reduces *URDF* size, e.g.

two arms (in *URDF*) generated from one macro (in *xacro*)

Introduction

Robot modeling

Manipulator

Pose, transformation

Elementary rotations

Representation of
rotation

Coordinate

Transformation

Direct kinematics

DH

ROS 2

integration

URDF

tf2

Semantic Robot Description Format (SRDF)★

Extends URDF with semantic description

- groups of joints and links
- kinematic chains
- end effectors
- named states

Introduction

Robot modeling

Manipulator

Pose, transformation

Elementary rotations

Representation of
rotation

Coordinate
Transformation

Direct kinematics

DH

ROS 2
integration

URDF

tf2

TF in ROS 2

TF is a system for keeping track of coordinate frames over time

Typical question:

$$T_B^A(t) = ?$$

- where is frame B with respect to frame A ?
- at time t ?
- how can be a vector transformed from one frame to another?

TF represents coordinate frames as a tree

Each edge is a transform:

$$T_{child}^{parent}$$

- one parent for each frame
- many nodes may publish different transforms (edges)
- TF can compute indirect transforms by chaining edges
- loops are not allowed

Example:

$$T_{camera}^{map} = T_{odom}^{map} T_{base_link}^{odom} T_{camera}^{base_link}$$

Dynamic transforms change over time

- published on `/tf`
- examples: mobile robot pose, moving links

Static transforms do not change

- published on `/tf_static`
- examples: frame of camera mounted on mobile robot base, fixed tool frame wrt. the end effector

Transform broadcaster

- publishes a transform between two frames
- usually publishes `geometry_msgs/TransformStamped`
- examples: odometry node, robot state publisher, localization node

Transform listener

- receives transforms and stores them in a buffer
- queries transforms between frames
- transforms points, poses, vectors, and sensor data

Typical query:

`lookupTransform(target frame, source frame,  $t$ )`

TF uses many-to-many *topic* architecture

A TF transform is time-dependent:

$$T_B^A(t)$$

Important rule:

- sensor data has a timestamp
- TF lookup should usually use the sensor timestamp
- not necessarily `now()`

Example:

$$T_{camera}^{map}(t_{image})$$

Common problems:

- missing transform
- transform too old
- extrapolation into the future
- mixed wall time and simulation time

Useful TF tools

Inspect TF frames:

```
ros2 run tf2_tools view_frames
```

Query one transform:

```
ros2 run tf2_ros tf2_echo map base_link
```

Check raw topics:

```
ros2 topic echo /tf
```

```
ros2 topic echo /tf_static
```

Publish a static transform:

```
ros2 run tf2_ros static_transform_publisher ...
```

```
rviz2
```

Introduction

Robot modeling

Manipulator

Pose, transformation

Elementary rotations

Representation of
rotation

Coordinate
Transformation

Direct kinematics

DH

ROS 2
integration

URDF

tf2

Questions?